1. Bonjour le monde
2. Commande d'écho
3. commentaires
4. Commentaire multiligne
5. Pendant que la boucle
6. Pour la boucle
7. Obtenir l'entrée de l'utilisateur
8. Si déclaration
9. Et Condition si déclaration
10. Ou Condition si déclaration
11. Sinon si et sinon condition
12. Condition du cas
13. Obtenir des arguments à partir de la ligne de commande
14. Obtenir des arguments à partir de la ligne de commande avec des noms
15. Combiner deux chaînes dans une variable
16. Obtenir une sous-chaîne de chaînes
17. Ajouter 2 nombres dans une variable
18. Créer une fonction
19. Utiliser les paramètres de fonction
20. Transmettre la valeur de retour du script

21. Créer un répertoire

22. Créer un répertoire en vérifiant l'existence

23. Lire un fichier

24. Supprimer un fichier

25. Ajouter au fichier

26. Tester si le fichier existe

27. Exemple d'envoi d'e-mail

28. Obtenir la date actuelle de l'analyse

29. Attendre la commande

30. Commande de sommeil

Créer et exécuter le premier programme BASH :

Vous pouvez exécuter un script bash à partir du terminal ou en exécutant n'importe quel fichier bash. Exécutez la commande suivante à partir du terminal pour exécuter une instruction bash très simple. La sortie de la commande sera « **Hello World** ».

```
$ echo "Bonjour tout le monde"
```

```
ubuntu@ubuntu-VirtualBox:~$ echo "Hello World"
Hello World
ubuntu@ubuntu-VirtualBox:~$
```

Ouvrez n'importe quel éditeur pour créer un fichier bash. Ici, l'éditeur **nano** est utilisé pour créer le fichier et le nom du fichier est défini sur " **First.sh** "

```
$ nano First.sh
```

Ajoutez le script bash suivant au fichier et enregistrez le fichier.

```
#!/bin/bash
echo "Hello World"
```

```
  GNU nano 2.8.6                    File: First.sh                    Modified

#!/bin/bash
echo "Hello World"


^G Get Help   ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos
^X Exit       ^R Read File  ^\ Replace    ^U Uncut Text^T To Linter   ^_ Go To Line
```

Vous pouvez exécuter le fichier bash de deux manières. Une façon consiste à utiliser la commande bash et une autre consiste à définir l'autorisation d'exécution sur le fichier bash et à exécuter le fichier. Les deux manières sont montrées ici.

```
$ bash First.sh
```

Ou,

```
$ chmod a+x First.sh
$ . / Premier.sh
```

```
ubuntu@ubuntu-VirtualBox:~$ bash First.sh
Hello World
ubuntu@ubuntu-VirtualBox:~$ chmod a+x First.sh
ubuntu@ubuntu-VirtualBox:~$ ./First.sh
Hello World
ubuntu@ubuntu-VirtualBox:~$
```

Utilisation de la commande echo :

Vous pouvez utiliser la commande echo avec diverses options. Certaines options utiles sont mentionnées dans l'exemple suivant. Lorsque vous utilisez la commande ' **echo** ' sans aucune option, une nouvelle ligne est ajoutée par défaut. **L'** option **'-n'** est utilisée pour imprimer n'importe quel texte sans nouvelle ligne et **l'** option **'-e'** est utilisée pour supprimer les barres obliques inverses de la sortie. Créez un nouveau fichier bash avec un nom, ' **echo_example.sh** ' et ajoutez le script suivant.

```
#!/bin/bash
echo "Impression de texte avec saut de ligne"
echo -n "Impression de texte sans saut de ligne"
echo -e " \n Suppression de \t barre oblique inverse \t caractères \n "
```

Exécutez le fichier avec la commande bash.

```
$ bash echo_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash echo_example.sh
Printing text with newline
Printing text without newline
Removing         backslash       characters

ubuntu@ubuntu-VirtualBox:~/code$
```

Utilisation du commentaire :

Le symbole **'#'** est utilisé pour ajouter un commentaire sur une seule ligne dans le script bash. Créez un nouveau fichier nommé ' **comment_example.sh'** et ajoutez le script suivant avec un commentaire sur une seule ligne.

```
#!/bin/bash

# Ajouter deux valeurs numériques
( ( sum = 25 + 35 ) )

#Imprimer le résultat
echo $sum
```

Exécutez le fichier avec la commande bash.

```
$ bash comment_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash comment_example.sh
60
ubuntu@ubuntu-VirtualBox:~/code$
```

Utilisation du commentaire multiligne :

Vous pouvez utiliser des commentaires sur plusieurs lignes dans bash de différentes manières. Un moyen simple est illustré dans l'exemple suivant. Créez un nouveau bash nommé **'multiline-comment.sh'** et ajoutez le script suivant. Ici, **les** symboles **':'** et **" ' "** sont utilisés pour ajouter un commentaire multiligne dans le script bash. Ce script suivant calculera le carré de 5.

```
#!/bin/bash
: '
Le script suivant calcule
la valeur carrée du nombre, 5.
'
( ( area = 5 * 5 ) )
echo $area
```

Exécutez le fichier avec la commande bash.

```
$ bash commentaire-multiligne.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash multiline-comment.sh
25
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez consulter le lien suivant pour en savoir plus sur l'utilisation du commentaire bash.

Utilisation de la boucle While :

Créez un fichier bash avec le nom, **« while_example.sh »,** à savoir l'utilisation de **tout** boucle. Dans l'exemple, **en** boucle pour itérer **5** fois. La valeur de la variable **count** augmentera de **1** à chaque étape. Lorsque la valeur de **comptage** variable 5 alors **en** boucle prendra fin.

```
#!/bin/bash
valid = true
count = 1
while [ $valid ]
do
echo $count
if [ $count -eq 5 ] ;
puis
casser
fi
( ( count++ ) )
fait
```

Exécutez le fichier avec la commande bash.

```
$ bash while_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash while_example.sh
1
2
3
4
5
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez consulter le lien suivant pour en savoir plus sur l'utilisation de la boucle while.

Utilisation de la boucle For :

La déclaration de boucle **for de** base est illustrée dans l'exemple suivant. Créez un fichier nommé « **for_example.sh** » et ajoutez le script suivant **à l'** aide **de la** boucle **for** . Ici, la boucle **for** itérera **10** fois et imprimera toutes les valeurs de la variable, **compteur** sur une seule ligne.

```
#!/bin/bash
for ( ( counter = 10 ; counter > 0 ; counter-- ) )
do
echo -n " $counter "
```

```
done
printf " \n "
```

Exécutez le fichier avec la commande bash.

```
$ bash pour_exemple.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash for_example.sh
10 9 8 7 6 5 4 3 2 1
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez utiliser la boucle for à différentes fins et de différentes manières dans votre script bash. Vous pouvez consulter le lien suivant pour en savoir plus sur l'utilisation de la boucle for.

Obtenir l'entrée de l'utilisateur :

La commande ' **read** ' est utilisée pour prendre l'entrée de l'utilisateur dans bash. Créez un fichier nommé ' **user_input.sh** ' et ajoutez le script suivant pour prendre l'entrée de l'utilisateur. Ici, une valeur de chaîne sera extraite de l'utilisateur et affichera la valeur en combinant une autre valeur de chaîne.

```
#!/bin/bash
echo "Entrez votre nom"
read name
echo "Bienvenue $name sur LinuxHint"
```

Exécutez le fichier avec la commande bash.

```
$ bash user_input.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash user_input.sh
Enter Your Name
Fahmida
Welcome Fahmida to LinuxHint
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez consulter le lien suivant pour en savoir plus sur l'utilisation des entrées utilisateur.

Utilisation de l'instruction if :

Vous pouvez utiliser la condition if avec une ou plusieurs conditions. Le bloc de début et de fin de cette instruction est défini par **'if'** et **'fi'** . Créez un fichier nommé ' **simple_if.sh** ' avec le script suivant pour connaître l' instruction use **if** dans bash. Ici, **10** est affecté à la variable **n** . si la valeur de **$n** est inférieure à 10, la sortie sera « **C'est un nombre à un chiffre** », sinon la sortie sera « **C'est un nombre à deux chiffres** ». À titre de comparaison, **'-lt'** est utilisé ici. A titre de comparaison, vous pouvez également utiliser **'-eq'** pour l' **égalité** , **'-ne'**pour **pas d'égalité** et **'-gt'** pour **plus grand que** dans le script bash.

```
#!/bin/bash
n = 10
if [ $n -lt 10 ] ;
then
echo "C'est un nombre à un chiffre"
sinon
echo "C'est un nombre à deux chiffres"
fi
```

Exécutez le fichier avec la commande bash.

```
$ bash simple_if.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash simple_if.sh
It is a two digit number
ubuntu@ubuntu-VirtualBox:~/code$
```

Utilisation de l'instruction if avec logique AND :

Différents types de conditions logiques peuvent être utilisés dans une instruction if avec deux ou plusieurs conditions. L'exemple suivant montre comment définir plusieurs conditions dans l'instruction if utilisant la logique **AND** . **'&&'** est utilisé pour appliquer la logique **AND** de l'instruction **if** . Créez un fichier nommé **'if_with_AND.sh'** pour vérifier le code suivant. Ici, la valeur des variables **nom** d' **utilisateur** et **mot de passe** sera extraite de l'utilisateur et comparée à ' **admin** ' et ' **secret** '. Si les deux valeurs correspondent, la sortie sera « **utilisateur valide** », sinon la sortie sera « **utilisateur invalide** ».

```
!/ bin / bash

echo "Entrez le nom d'utilisateur"
read username
echo "Entrez le mot de passe"
lit le mot de passe

if [ [ ( $username == "admin" && $password == "secret" ) ] ] ; then
echo "utilisateur valide"
else
echo "utilisateur invalide"
fi
```

Exécutez le fichier avec la commande bash.

```
$ bash if_with_AND.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash if_with_AND.sh
Enter username
admin
Enter password
1234
invalid user
ubuntu@ubuntu-VirtualBox:~/code$ bash if_with_AND.sh
Enter username
admin
Enter password
secret
valid user
ubuntu@ubuntu-VirtualBox:~/code$
```

Utilisation de l'instruction if avec logique OR :

' **||** ' est utilisé pour définir la logique **OU** dans la condition **if** . Créez un fichier nommé **'if_with_OR.sh'** avec le code suivant pour vérifier l'utilisation de la logique **OR** de l' instruction **if** . Ici, la valeur de **n** sera prise par l'utilisateur. Si la valeur est égale à **15** ou **45** alors la sortie sera « **Vous avez gagné la partie** », sinon la sortie sera « **Vous avez perdu la partie** ».

```
#!/bin/bash

echo "Entrez n'importe quel nombre"
read n

if [ [ ( $n -eq 15 || $n   -eq 45 ) ] ]
then
echo "Vous avez gagné la partie"
else
echo "Vous avez perdu la partie"
fi
```

Exécutez le fichier avec la commande bash.

```
$ bash if_with_OR.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash if_with_OR.sh
Enter any number
40
You lost the game
ubuntu@ubuntu-VirtualBox:~/code$ bash if_with_OR.sh
Enter any number
15
You won the game
ubuntu@ubuntu-VirtualBox:~/code$
```

Utilisation de l'instruction else if :

L'utilisation de la condition **else if** est peu différente dans bash par rapport aux autres langages de programmation. ' **elif** ' est utilisé pour définir

la condition **else if** dans bash. Créez un fichier nommé ' **elseif_example.sh** ' et ajoutez le script suivant pour vérifier comment **else if** est défini dans le script bash.

```
#!/bin/bash

echo "Entrez votre numéro porte-bonheur"
read n

if [ $n -eq 101 ] ;
then
echo "Vous avez obtenu le 1er prix"
elif [ $n -eq 510 ] ;
then
echo "Vous avez obtenu le 2e prix"
elif [ $n -eq 999 ] ;
then
echo "Vous avez le 3e prix"

else
echo "Désolé, essayez pour la prochaine fois"
fi
```

Exécutez le fichier avec la commande bash.

```
$ bash elseif_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash elseif_example.sh
Enter your lucky number
101
You got 1st prize
ubuntu@ubuntu-VirtualBox:~/code$ bash elseif_example.sh
Enter your lucky number
999
You got 3rd prize
ubuntu@ubuntu-VirtualBox:~/code$ bash elseif_example.sh
Enter your lucky number
100
Sorry, try for the next time
ubuntu@ubuntu-VirtualBox:~/code$
```

Utilisation de la déclaration de cas :

L' instruction **Case** est utilisée comme alternative à l'instruction **if-elseif-else** . Le bloc de début et de fin de cette instruction est défini par ' **case** ' et ' **esac** '. Créez un nouveau fichier nommé ' **case_example.sh** ' et ajoutez le script suivant. La sortie du script suivant sera identique au précédent **else if** example.

```
#!/bin/bash

echo "Entrez votre numéro porte-bonheur"
```

```
read n
case $n in
101 )
echo echo "Vous avez le 1er prix" ;;
510 )
echo "Vous avez obtenu le 2ème prix" ;;
999 )
echo "Vous avez obtenu le 3ème prix" ;;
* )
echo "Désolé, essayez pour la prochaine fois" ;;
esac
```

Exécutez le fichier avec la commande bash.

```
$ bash case_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash case_example.sh
Enter your lucky number
101
echo You got 1st prize
ubuntu@ubuntu-VirtualBox:~/code$ bash case_example.sh
Enter your lucky number
510
You got 2nd prize
ubuntu@ubuntu-VirtualBox:~/code$ bash case_example.sh
Enter your lucky number
999
You got 3rd prize
ubuntu@ubuntu-VirtualBox:~/code$ bash case_example.sh
Enter your lucky number
777
Sorry, try for the next time
ubuntu@ubuntu-VirtualBox:~/code$
```

Obtenez des arguments à partir de la ligne de commande :

Le script Bash peut lire l'entrée de l'argument de la ligne de commande comme tout autre langage de programmation. Par exemple, les variables **$1** et **$2** sont utilisées pour lire les premier et deuxième arguments de la ligne de commande. Créez un fichier nommé « **command_line.sh** » et ajoutez le script suivant. Deux valeurs d'argument lues par le script suivant et imprime le nombre total d'arguments et les valeurs d'argument en sortie.

```
#!/bin/bash
echo "Total des arguments : $#"
echo "1er argument = $1"
echo "2e argument = $2"
```

Exécutez le fichier avec la commande bash.

```
$ bash command_line.sh Indice Linux
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash command_line.sh Linux Hint
Total arguments : 2
1st Argument = Linux
2nd argument = Hint
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez consulter le lien suivant pour en savoir plus sur l'utilisation de l'argument de ligne de commande.

Obtenez des arguments à partir de la ligne de commande avec des noms :

La façon dont vous pouvez lire les arguments de ligne de commande avec des noms est indiquée dans le script suivant. Créez un fichier nommé ' **command_line_names.sh'** et ajoutez le code suivant. Ici, deux arguments, **X** et **Y** sont lus par ce script et impriment la somme de X et Y.

```
#!/bin/bash
for arg in " $@ "
do
index =$ ( echo $arg | cut -f1 -d = )
val =$ ( echo $arg | cut -f2 -d = )
case $index in
X ) x = $val ;;

Y ) y = $val ;;

* )
esac
fait
( ( résultat =x+y ) )
echo "X+Y= $résultat "
```

Exécutez le fichier avec la commande bash et avec deux arguments de ligne de commande.

```
$ bash command_line_names X = 45 Y = 30
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash command_line_names.sh X=45 Y=30
X+Y=75
ubuntu@ubuntu-VirtualBox:~/code$
```

Combinez les variables de chaîne :

Vous pouvez facilement combiner des variables de chaîne dans bash. Créez un fichier nommé « **string_combine.sh** » et ajoutez le script suivant pour vérifier comment vous pouvez combiner des variables de chaîne dans bash en plaçant des variables ensemble ou en utilisant **l'** opérateur **'+'** .

```
#!/bin/bash
```

```
string1 = "Linux"
string2 = "Hint"
echo " $string1 $
string2 " string3 = $string1 + $
string2 string3+= " est un bon site de blog tutoriel"
echo $string3
```

Exécutez le fichier avec la commande bash.

```
$ bash string_combine.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash string_combine.sh
LinuxHint
Linux+Hint is a good tutorial blog site
ubuntu@ubuntu-VirtualBox:~/code$
```

Obtenir la sous-chaîne de String :

Comme les autres langages de programmation, bash n'a pas de fonction intégrée pour couper la valeur des données de chaîne. Mais vous pouvez effectuer la tâche de sous-chaîne d'une autre manière dans bash, comme indiqué dans le script suivant. Pour tester le script, créez un fichier nommé ' **substring_example.sh** ' avec le code suivant. Ici, la valeur **6** indique le point de départ à partir duquel la sous-chaîne commencera et **5** indique la longueur de la sous-chaîne.

```
#!/bin/bash
Str = "Apprendre Linux à partir de LinuxHint"
subStr = ${Str:6:5}
echo $subStr
```

Exécutez le fichier avec la commande bash.

```
$ bash sous-chaîne_exemple.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash substring_example.sh
Linux
ubuntu@ubuntu-VirtualBox:~/code$
```

Ajouter deux nombres :

Vous pouvez effectuer les opérations arithmétiques dans bash de différentes manières. La façon dont vous pouvez ajouter deux nombres entiers dans bash à l'aide de doubles crochets est illustrée dans le script suivant. Créez un fichier nommé ' **add_numbers.sh** ' avec le code suivant. Deux valeurs entières seront extraites de l'utilisateur et imprimées le résultat de l'addition.

```
#!/bin/bash
echo "Entrez le premier nombre"
read x
echo "Entrez le deuxième nombre"
```

```
read y
( ( sum =x+y ) )
echo "Le résultat de l'addition= $sum "
```

Exécutez le fichier avec la commande bash.

```
$ bash add_numbers.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash add_numbers.sh
Enter first number
25
Enter second number
56
The result of addition=81
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez consulter le lien suivant pour en savoir plus sur l'arithmétique bash.

Créer une fonction :

La façon dont vous pouvez créer une fonction simple et appeler la fonction est illustrée dans le script suivant. Créez un fichier nommé ' **function_example.sh** ' et ajoutez le code suivant. Vous pouvez appeler n'importe quelle fonction par son nom uniquement sans utiliser de crochet dans le script bash.

```
#!/bin/bash
function F1 ( )
{
echo 'J'aime la programmation bash'
}

F1
```

Exécutez le fichier avec la commande bash.

```
$ bash function_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash function_example.sh
I like bash programming
ubuntu@ubuntu-VirtualBox:~/code$
```

Créer une fonction avec des paramètres :

Bash ne peut pas déclarer de paramètre de fonction ou d'arguments au moment de la déclaration de fonction. Mais vous pouvez utiliser des paramètres dans la fonction en utilisant une autre variable. Si deux valeurs sont transmises au moment de l'appel de la fonction, les variables $1 et $2 sont utilisées pour lire les valeurs. Créez un fichier nommé ' **function|_parameter.sh** ' et ajoutez le code suivant. Ici, la fonction ' **Rectangle_Area'** calculera l'aire d'un rectangle en fonction des valeurs des paramètres.

```
#!/bin/bash

Rectangle_Area ( ) {
area =$ ( ( $1 * $2 ) )
echo "La zone est : $area "
}

Rectangle_Area 10 20
```

Exécutez le fichier avec la commande bash.

```
$ bash fonction_paramètre.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash function_parameter.sh
Area is : 200
ubuntu@ubuntu-VirtualBox:~/code$
```

Passer la valeur de retour de la fonction :

La fonction Bash peut transmettre à la fois des valeurs numériques et des valeurs de chaîne. L'exemple suivant montre comment vous pouvez passer une valeur de chaîne à partir de la fonction. Créez un fichier nommé " **function_return.sh** " et ajoutez le code suivant. La fonction, **Greeting()** renvoie une valeur de chaîne dans la variable, **val** qui s'imprime plus tard en la combinant avec une autre chaîne.

```
#!/bin/bash
function salutation ( ) {

str = "Bonjour, $nom "
echo $str

}

echo "Entrez votre nom"
read name

val =$ ( salutation )
echo "La valeur de retour de la fonction est $val "
```

Exécutez le fichier avec la commande bash.

```
$ bash fonction_retour.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash function_return.sh
Enter your name
John
Return value of the function is Hello, John
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez consulter le lien suivant pour en savoir plus sur l'utilisation de la fonction bash.

Créer un répertoire :

Bash utilise la commande ' **mkdir** ' pour créer un nouveau répertoire. Créez un fichier nommé ' **make_directory.sh** ' et ajoutez le code suivant pour prendre un nouveau nom de répertoire de l'utilisateur. Si le nom du répertoire n'existe pas dans l'emplacement actuel, il créera le répertoire, sinon le programme affichera une erreur.

```
#!/bin/bash
echo "Entrez le nom du répertoire"
read newdir
` mkdir $newdir `
```

Exécutez le fichier avec la commande bash.

```
$ bash make_directory.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code/temp$ bash make_directory.sh
Enter directory name
test_dir
ubuntu@ubuntu-VirtualBox:~/code/temp$ ls
make_directory.sh  test_dir
ubuntu@ubuntu-VirtualBox:~/code/temp$
```

Créer un répertoire en vérifiant l'existence :

Si vous souhaitez vérifier l'existence du répertoire à l'emplacement actuel avant d'exécuter la commande ' **mkdir** ', vous pouvez utiliser le code suivant. **L'** option **'-d** ' est utilisée pour tester qu'un répertoire particulier existe ou non. Créez un fichier nommé ' **directory_exist.sh'** et ajoutez le code suivant pour créer un répertoire en vérifiant son existence.

```
#!/bin/bash
echo "Entrez le nom du répertoire"
read ndir
if [ -d " $ndir " ]
then
echo "Le répertoire existe"
else
` mkdir $ndir `
echo "Répertoire créé"
fi
```

Exécutez le fichier avec la commande bash.

```
$ bash répertoire_exist.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code/temp$ bash directory_exist.sh
Enter directory name
newdir
Directory created
ubuntu@ubuntu-VirtualBox:~/code/temp$ ls
directory_exist.sh  make_directory.sh  newdir  test_dir
ubuntu@ubuntu-VirtualBox:~/code/temp$
```

Vous pouvez consulter le lien suivant pour en savoir plus sur la création de répertoires.

Lire un fichier :

Vous pouvez lire n'importe quel fichier ligne par ligne dans bash en utilisant loop. Créez un fichier nommé « **read_file.sh** » et ajoutez le code suivant pour lire un fichier existant nommé « **book.txt** ».

```
#!/bin/bash
file = 'book.txt'
lors de la lecture de la ligne ; faire
echo $line
done < $file
```

Exécutez le fichier avec la commande bash.

```
$ bash read_file.sh
```

Exécutez la commande suivante pour vérifier le contenu original du fichier « **book.txt** ».

```
$ chat livre.txt
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash read_file.sh
1. Pro AngularJS
2. Learning JQquery
3. PHP Programming
4. CodeIgniter 3
ubuntu@ubuntu-VirtualBox:~/code$ cat book.txt
1. Pro AngularJS
2. Learning JQquery
3. PHP Programming
4. CodeIgniter 3
ubuntu@ubuntu-VirtualBox:~/code$
```

Vous pouvez consulter le lien suivant pour connaître les différentes manières de lire le fichier.

https://linuxhint.com/read_file_line_by_line_bash/

Aller en haut

Supprimer un fichier :

La commande " **rm** " est utilisée dans bash pour supprimer n'importe quel fichier. Créez un fichier nommé ' **delete_file.sh** ' avec le code suivant pour prendre le nom de fichier de l'utilisateur et le supprimer. Ici, **l'** option **'-i'** est utilisée pour obtenir l'autorisation de l'utilisateur avant de supprimer le fichier.

```
#!/bin/bash
echo "Entrez le nom du fichier à supprimer"
read fn
rm -i $fn
```

Exécutez le fichier avec la commande bash.

```
$ ls
$ bash delete_file.sh
$ ls
```

```
ubuntu@ubuntu-VirtualBox:~/code/temp$ ls
delete_file.sh      make_directory.sh  test_dir
directory_exist.sh  newdir             test.txt
ubuntu@ubuntu-VirtualBox:~/code/temp$ bash delete_file.sh
Enter filename to remove
test.txt
rm: remove regular file 'test.txt'? y
ubuntu@ubuntu-VirtualBox:~/code/temp$ ls
delete_file.sh  directory_exist.sh  make_directory.sh  newdir  test_dir
ubuntu@ubuntu-VirtualBox:~/code/temp$
```

Ajouter au fichier :

De nouvelles données peuvent être ajoutées dans n'importe quel fichier existant en utilisant **l'** opérateur **'>>'** dans bash. Créez un fichier nommé **« append_file.sh** » et ajoutez le code suivant pour ajouter un nouveau contenu à la fin du fichier. Ici, ' **Learning Laravel 5** ' sera ajouté au fichier ' **book.txt'** après l'exécution du script.

```
#!/bin/bash

echo "Avant d'ajouter le fichier"
cat book.txt

echo "Apprentissage de Laravel 5" >> book.txt
echo "Après avoir ajouté le fichier"
cat book.txt
```

Exécutez le fichier avec la commande bash.

```
$ bash fichier_append.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code/temp$ bash append_file.sh
Before appending the file
1. Pro AngularJS
2. Learning JQquery
3. PHP Programming
4. CodeIgniter 3
After appending the file
1. Pro AngularJS
2. Learning JQquery
3. PHP Programming
4. CodeIgniter 3
Learning Laravel 5
ubuntu@ubuntu-VirtualBox:~/code/temp$
```

Tester si le fichier existe :

Vous pouvez vérifier l'existence du fichier dans bash en utilisant **l'** option **'-e'** ou **'-f'** . **L'** option **'-f'** est utilisée dans le script suivant pour tester l'existence du fichier. Créez un fichier nommé « **file_exist.sh** » et ajoutez le code suivant. Ici, le nom du fichier passera depuis la ligne de commande.

```
#!/bin/bash
filename = $1
if [ -f " $filename " ] ; then
echo "Le fichier existe"
else
echo "Le fichier n'existe pas"
fi
```

Exécutez les commandes suivantes pour vérifier l'existence du fichier. Ici, le fichier **book.txt** existe et **book2.txt** n'existe pas à l'emplacement actuel.

```
$ ls
$ bash file_exist.sh book.txt
$ bash file_exist.sh book2.txt
```

```
ubuntu@ubuntu-VirtualBox:~/code/temp$ ls
append_file.sh  delete_file.sh      file_exist.sh      newdir
book.txt        directory_exist.sh  make_directory.sh  test_dir
ubuntu@ubuntu-VirtualBox:~/code/temp$ bash file_exist.sh book.txt
File exists
ubuntu@ubuntu-VirtualBox:~/code/temp$ bash file_exist.sh book2.txt
File does not exist
ubuntu@ubuntu-VirtualBox:~/code/temp$
```

Envoyer un e-mail :

Vous pouvez envoyer un e-mail en utilisant la commande ' **mail** ' ou ' **sendmail** '. Avant d'utiliser ces commandes, vous devez installer tous les packages nécessaires. Créez un fichier nommé « **mail_example.sh** » et ajoutez le code suivant pour envoyer l'e-mail.

```
#!/bin/bash
Recipient =”admin @ example.com”
Subject =”Greeting”
Message =”Bienvenue sur notre site”
` mail -s $Subject $Recipient <<< $Message `
```

Exécutez le fichier avec la commande bash.

```
$ bash mail_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash mail_example.sh
ubuntu@ubuntu-VirtualBox:~/code$
```

Obtenir la date actuelle de l'analyse :

Vous pouvez obtenir la date et l'heure actuelles du système à l'aide de la commande ` **date** `. Chaque partie de la valeur de date et d'heure peut être analysée à l'aide de ' **Y', 'm', 'd', 'H', 'M'** et ' **S'** . Créez un nouveau fichier nommé « **date_parse.sh »** et ajoutez le code suivant pour séparer les valeurs du jour, du mois, de l'année, de l'heure, des minutes et des secondes.

```
#! / bin / bash
année = ` jour + % Y `
Mois = ` jour + % m `
jour = ` jour + % d `
heure = ` jour + % H `
minute = ` jour + % M `
Deuxième = ` jour + % S `
echo ` date `
echo "La date actuelle est :$Day - $Month - $Year "
echo "L'heure actuelle est : $Hour : $Minute : $Second "
```

Exécutez le fichier avec la commande bash.

```
$ bash date_parse.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash date_parse.sh
Mon Jul 9 12:04:41 EDT 2018
Current Date is: 09-07-2018
Current Time is: 12:04:41
ubuntu@ubuntu-VirtualBox:~/code$
```

Commande d'attente :

wait est une commande intégrée de Linux qui attend la fin de tout processus en cours. **La** commande **wait** est utilisée avec un identifiant de processus ou un identifiant de travail particulier. Si aucun ID de processus ou d'ID de travail n'est donné avec la commande wait, il attendra que tous les processus enfants actuels se terminent et renvoie l'état de sortie. Créez un fichier nommé ' **wait_example.sh'** et ajoutez le script suivant.

```
#!/bin/bash
echo "Wait command" &
process_id = $!
wait $process_id
echo "Sortie avec le statut $?"
```

Exécutez le fichier avec la commande bash.

```
$ bash wait_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash wait_example.sh
Wait command
Exited with status 0
```

Vous pouvez consulter le lien suivant pour en savoir plus sur la commande wait.

Commande de sommeil :

Lorsque vous souhaitez suspendre l'exécution d'une commande pendant une période spécifique, vous pouvez utiliser **la** commande **sleep** . Vous pouvez régler le délai en **secondes (s), minutes (m), heures (h) et jours (d).** Créez un fichier nommé **'sleep_example.sh'** et ajoutez le script suivant. Ce script attendra 5 secondes après l'exécution.

```
#! / bin / bash

echo « Attendez pour 5 secondes »
sommeil 5
echo « Terminé »
```

Exécutez le fichier avec la commande bash.

```
$ bash sleep_example.sh
```

```
ubuntu@ubuntu-VirtualBox:~/code$ bash sleep_example.sh
Wait for 5 seconds
Completed
ubuntu@ubuntu-VirtualBox:~/code$
```